Vietnam Journal of Computer Science
© World Scientific Publishing Company

# GA-MO: Pareto-Optimal Split Computing for Deep Edge Intelligence

Nguyen Tan Y
*Faculty of Engineering and Technology*
*University of Phan Thiet*
*Phan Thiet, Lam Dong Vietnam*

Le Hoang Trung
*Research Engineer at LG Electronics R&D*
*Da Nang, Vietnam*

Tran Anh Khoa
*Modeling Evolutionary Algorithms Simulation and Artificial Intelligence*
*Faculty of Electrical and Electronics Engineering, Ton Duc Thang University*
*Ho Chi Minh city, Vietnam*

Nguyen Hoang Nam*
*Modeling Evolutionary Algorithms Simulation and Artificial Intelligence*
*Faculty of Electrical and Electronics Engineering, Ton Duc Thang University*
*Ho Chi Minh city, Vietnam*

Deploying deep neural networks (DNNs) on resource-constrained devices requires hybrid device–cloud inference in which end-to-end latency and data transmission must be optimized jointly. Existing Dynamic Split Computing (DSC) methods primarily target latency and overlook the computation–communication trade-off; as a result, varying bandwidth and batch size can lead to suboptimal operating points (e.g., excessive transmission for marginal latency gains). We propose Genetic Algorithm Multi-Objective (GA-MO), a genetic-algorithm framework that constructs Pareto fronts across diverse bandwidth scenarios and supports adaptive runtime selection. GA-MO jointly reduces inference latency and transmission load to satisfy application-specific performance requirements, including common real-time constraints. Experiments on EfficientNet-B0 and VGG16 show that GA-MO outperforms DSC, achieving 55–60% lower latency and 52–62% bandwidth savings, with a 100% win rate across all tested conditions. These results indicate that GA-MO is an effective and scalable approach to multi-objective optimization for hybrid edge–cloud inference.

*Keywords*: Genetic Algorithm, Split Computing, Edge–Cloud, Latency Reduction, Bandwidth Optimization, Edge Intelligence

*Corresponding author: Dr. Nguyen Hoang Nam: nguyenhoangnam@tdtu.edu.vn.

## 1. Introduction

## 2. Introduction

Federated Learning (FL) enables distributed training of deep neural networks (DNNs) by allowing clients to collaboratively update models without transmitting raw data.[1] FL has found applications in healthcare, finance, and edge intelligence.[2,3,5] However, conventional FL assumes that each client can host and train a complete DNN locally—an assumption that becomes impractical as models and datasets scale, while client devices remain resource constrained. For modern architectures, the computational and memory demands often exceed the capabilities of mobile, IoT, and embedded platforms.

To alleviate these limitations, *split learning/split computing* (SC) has been introduced as a complementary paradigm to FL.[4–8] In SC, a network is divided into a client-side and a server-side component: the client processes inputs up to a designated split layer and transmits the intermediate features to the server, which completes the remaining computation. This strategy reduces the processing and memory burden on the device, enabling practical device–cloud deployment.[9] SC also broadens client participation, supports models larger than device capacity, and mitigates privacy concerns since raw inputs and labels are not exposed.[6]

Despite these benefits, SC faces several challenges. First, split points are often chosen manually or with simple heuristics, overlooking dynamic factors such as fluctuating bandwidth and heterogeneous devices. Second, most prior work evaluates moderately sized networks (e.g., VGG16), leaving large-scale modern architectures insufficiently explored. Third, systematic methods for *jointly* optimizing split placement to minimize end-to-end latency—by balancing device computation with network transmission—remain limited. Recent surveys highlight automated and adaptive split strategies as an important but underdeveloped direction.[6,7]

Dynamic Split Computing (DSC) has been proposed to enable adaptive split-point selection.[8,19] However, existing methods largely rely on instantaneous measurements (e.g., channel state, batch size). Such approaches depend on precise real-time monitoring and ignore historical system behavior, limiting adaptability across operating conditions. More importantly, most DSC strategies optimize a *single* objective—typically latency—while neglecting the computation–communication trade-off. Consequently, under varying bandwidth $r$ and batch size $b$, DSC may become inefficient, either transmitting excessive data for marginal latency gains or losing efficiency in the opposite direction.

To address these limitations, we present **Genetic Algorithm Multi-Objective (GA-MO)**,[11,12] an evolutionary framework that explicitly optimizes both inference latency and communication cost. By systematically exploring candidate configurations across split points, quantization levels, and compression ratios, GA-MO constructs Pareto fronts that capture the full spectrum of latency–bandwidth trade-offs, allowing practitioners to select runtime operating points that align with application requirements.

The main contributions of this paper are as follows:

- We reformulate split computing as a **multi-objective optimization** problem that jointly minimizes inference latency and transmission cost, addressing the single-objective limitation of DSC.
- We develop **GA-MO**, a genetic algorithm–based framework that efficiently approximates Pareto fronts across diverse bandwidth scenarios, accounting for batch-size scaling, per-MB overheads (compression, decompression, quantization/dequantization), and heterogeneous edge–cloud execution speeds.
- We conduct comprehensive simulations on **EfficientNet-B0** and **VGG16** across 30 bandwidth levels and 30 batch sizes. GA-MO consistently outperforms DSC, achieving **55–60% lower latency** and **52–62% bandwidth savings**, with a **100% win rate** across all tested conditions, while also delivering superior Pareto-front quality via hypervolume improvement.

The remainder of the paper is organized as follows. Section 3 reviews related studies. Section 4 describes the problem formulation and the proposed GA-MO framework. Section 5 presents the experimental results. Section 6 concludes the paper and discusses future research directions.

## 3. Related Works

Split computing has gained increasing attention as a strategy to distribute deep neural network (DNN) inference between edge devices and cloud servers, aiming to reduce latency and bandwidth cost while preserving accuracy and privacy. A prominent line of research is *Dynamic Split Computing* (DSC), in which partition points are selected adaptively according to system conditions. For instance,[8] proposed exploiting natural bottlenecks in DNNs to determine split points without retraining or hyperparameter tuning, enabling faster inference and improved privacy by transmitting intermediate activations instead of raw inputs.

On the benchmarking side,[13] introduced the SC2 Benchmark, a standardized platform for evaluating supervised feature compression in split computing, formaliz-

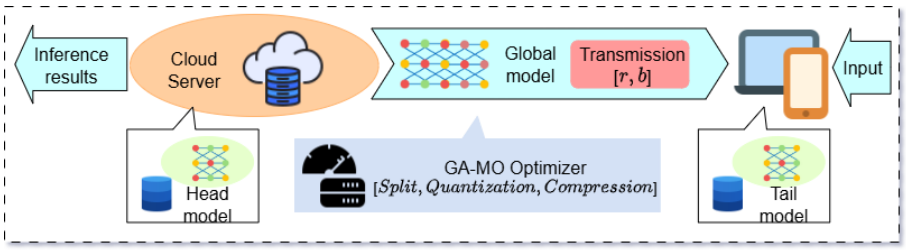



Fig. 1: Overview of split learning approach.

ing three objectives: minimizing device computation, reducing feature transmission size, and maximizing accuracy. Building on this,[14] presented Ladon, the first multi-task compression model supporting classification, detection, and segmentation in a unified pipeline, achieving latency reductions up to 95.4% and energy savings of 88.2%.

Recent work has explored automated architecture design for split computing. Neural Architecture Search (NAS) has been applied to jointly optimize architectures and split points, reducing latency by 40–60% with negligible accuracy loss.[15] Beyond conventional DNNs, split computing has been extended to spiking neural networks (SNNs). For example,[16] proposed Neuromorphic Wireless Split Computing, which transmits multi-level spike payloads over OFDM channels to balance communication and computation in neuromorphic systems.

Domain-specific applications have also emerged.[17] introduced ReinDSplit, a reinforcement learning–based framework for pest recognition in precision agriculture, demonstrating improved efficiency compared to static methods. Similarly, EC-SNN[18] partitions spiking neural networks across multiple devices, integrating channel pruning and parallel execution to reduce inference latency by 60.7% and energy by 27.7%. A Genetic Algorithm (GA)[12] has been applied to treat split-point selection as an optimization problem. By evolving candidate configurations with fitness functions derived from computation and transmission costs, GA achieves markedly higher efficiency than brute-force search (74.4% vs. 28.7%). This approach enables scalable and adaptive deployment of deep learning models under diverse device and network constraints.[19]

To improve scalability, adaptive frameworks such as AdaSplit[20] dynamically adjust split points and communication strategies in response to system conditions, although they still depend on heuristics and lack validation under highly dynamic environments. Finally, with the rise of 6G and distributed edge intelligence, Split Edge Learning (SEL) has been identified as a key paradigm for resource-efficient AI at the wireless edge,[21] though open challenges remain in convergence, asynchronous training, and coordination among distributed nodes.

In summary, prior work demonstrates rapid progress in split computing, ranging from adaptive partitioning and supervised compression to neuromorphic extensions and domain-specific applications. Nevertheless, most approaches remain driven by latency-oriented heuristics and do not explicitly optimize the joint computation–communication trade-off. This gap motivates our proposal of GA-MO, which formulates split computing as a multi-objective optimization problem and applies evolutionary search to efficiently approximate Pareto-optimal solutions.

## 4. Problem Formulation and GA-MO Method

### 4.1. *Problem Formulation*

Deploying deep neural networks (DNNs) in hybrid edge–cloud environments requires making careful decisions about how to partition the model and how to transmit in-

termediate features. These decisions are directly associated with two competing factors: the *end-to-end inference latency* and the *communication overhead*. Conventional Dynamic Split Computing (DSC) methods are typically designed to optimize only latency, while leaving communication cost unaddressed. In practice, however, focusing on a single objective often results in inefficient solutions: reducing latency alone may incur prohibitively large transmission costs, whereas reducing bytes alone may lead to unacceptable delays. To overcome this limitation, we reformulate split computing as a multi-objective optimization problem and propose GA-MO, a genetic algorithm framework designed to construct Pareto-optimal trade-offs between latency and communication.

Formally, each candidate configuration is represented as a tuple

$$x = (s, q, c, b), \tag{1}$$

where $s$ denotes the split point (layer/block) between edge and cloud, $q$ represents the quantization level (e.g., $q = 8$ bits for INT8), $c \in (0, 1]$ is the compression ratio of feature maps (with $c = 0.25$ corresponding to 25% of the original size), and $b$ is the batch size. In addition, the system is characterized by the effective bandwidth $r$, expressed in b/s or MB/s depending on the network condition.

Based on this formulation, the transmission cost in bytes can be modeled as

$$B(x) = A(s) \times \frac{q}{32} \times c \times b, \tag{2}$$

where $A(s)$ denotes the activation size (in float32) at the split point $s$.

Given this transmission cost, the corresponding transmission time is expressed as

$$T_{net}(x; r) = \frac{B(x)}{r} + \alpha, \tag{3}$$

where $\alpha$ represents a constant communication overhead that accounts for factors such as connection setup and protocol processing, independent of the actual payload size.

Consequently, the overall end-to-end latency can be written as

$$L(x; r) = T_{edge}(x) + T_{cloud}(x) + T_{net}(x; r) + T_{sched}, \tag{4}$$

where $T_{edge}$ and $T_{cloud}$ are the execution times at the device and the cloud, respectively, and $T_{sched}$ accounts for the scheduling overhead.

Therefore, the optimization problem is defined as

$$\min_{x \in \mathcal{X}} \big(L(x; r),\ B(x)\big), \quad \text{for a fixed } r, \tag{5}$$

where $\mathcal{X}$ denotes the feasible configuration space. For each bandwidth $r$, solving this problem yields a Pareto set $F(r)$, which characterizes the trade-off surface between inference latency and communication cost.

### 4.2. *GA-MO Method*

Considering the split computing formulation as a multi-objective optimization problem in Section 4.1. In Section 4.2, we now introduce GA-MO, a genetic algorithm specifically designed to approximate the Pareto front $F(r)$ under varying bandwidth conditions. The central idea is to avoid an exhaustive search, which is computationally prohibitive, and instead employ an evolutionary strategy that iteratively refines a population of candidate solutions.

In GA-MO, each individual encodes a configuration $x = (s, q, c, b)$, which consists of the split point $s$, quantization level $q$, compression ratio $c$, and batch size $b$. The fitness of each candidate is evaluated by computing both objectives, namely the end-to-end latency $L(x; r)$ and the transmission cost $B(x)$, as defined in Equations 2–4. Because the two objectives are conflicting, no single solution can minimize both simultaneously. Therefore, GA-MO employs the principle of *Pareto dominance*, whereby a solution is preferred if it is strictly better in at least one objective and not worse in the other.

The algorithm begins with a randomly generated population of candidate configurations. At each generation, the population is ranked using a non-dominated sorting procedure that organizes individuals into successive Pareto layers. The first layer corresponds to the current approximation of the Pareto front, while subsequent layers contain dominated solutions of decreasing quality. To preserve diversity along the trade-off surface, GA-MO incorporates a crowding-distance measure, ensuring that individuals are distributed across different regions of the latency–bandwidth spectrum.

After ranking, a subset of promising solutions is selected to produce offspring. The variation operators are then applied: crossover recombines two parent solutions to explore new combinations of split points, quantization levels, compression ratios, and batch sizes, while mutation introduces small perturbations to avoid premature convergence. Discrete parameters such as split point and quantization are perturbed by categorical sampling, whereas continuous parameters such as compression ratio and batch size are modified by bounded random variation.

The newly generated offspring are subsequently evaluated using the same latency and transmission models and then merged with the parent population. Non-dominated sorting is performed again, and only the top $N$ individuals are retained for the next generation. This process repeats until either a predefined number of generations has elapsed or the hypervolume indicator of the Pareto set converges, indicating that further iterations are unlikely to yield substantial improvements.

Finally, GA-MO outputs a Pareto front $F(r)$ for each bandwidth $r$. This Pareto set provides system designers with a menu of efficient operating points. Depending on the application, a configuration that minimizes latency, maximizes bandwidth savings, or balances both may be chosen. In this way, GA-MO offers a principled and flexible solution that extends beyond the limitations of single-objective DSC approaches.

**Algorithm 4.1** GA-MO: Multi-Objective Genetic Algorithm for Split Computing

**Input:** Configuration space $\mathcal{X}$; bandwidth $r$; population size $N$; max generations $T$.

**Output:** Pareto front $F(r)$ of non-dominated solutions.

Initialize population $P_0$ by sampling $N$ configurations $x \in \mathcal{X}$

**foreach** $x \in P_0$ **do**

  evaluate $f(x) = (L(x; r), B(x))$

$t \leftarrow 0$ **while** $t < T$ *and not converged* **do**

  Rank $P_t$ using non-dominated sorting

  Select promising candidates using Pareto rank and crowding distance

  Apply crossover and mutation to generate offspring $Q$

  Evaluate all $f(x)$ for $x \in Q$

  $P_{t+1} \leftarrow$ best $N$ individuals from $P_t \cup Q$

  $t \leftarrow t + 1$

**return** $F(r) =$ *non-dominated set from* $P_t$

### 4.3. *Evaluation Metrics*

To rigorously compare GA-MO with the DSC baseline, we employ a set of metrics that capture latency, communication efficiency, and Pareto front quality. These are grounded in split computing models,[8] multi-objective optimization practices,[22,23] and standard performance evaluation methodology.[24] Together, they enable point-wise and statistical comparisons across bandwidths and batch sizes.

**Latency improvement.** The latency gain is measured as

$$\Delta L(r, b) = L^{\star}_{GA\text{-}MO}(r, b) - L_{DSC}(r, b), \tag{6}$$

where $L^{\star}_{GA\text{-}MO}$ is the minimal latency solution on the Pareto set $F(r)$. Negative values indicate that GA-MO outperforms DSC.

**Byte Saving Ratio (BSR).** Communication efficiency is quantified by

$$BSR(r, b) = 1 - \frac{B^{\star}_{GA\text{-}MO}(r, b)}{B_{DSC}(r, b)}, \tag{7}$$

where $B^{\star}_{GA\text{-}MO}$ and $B_{DSC}$ denote transmitted bytes under GA-MO and DSC, respectively.

**Win-rate and $\varepsilon$-win.** We further report the fraction of cases in which GA-MO improves over DSC:

$$\text{WinRate} = \frac{1}{|\mathcal{G}|} \sum_{(r,b)\in\mathcal{G}} \mathbf{1}[\Delta L(r, b) \leq 0], \tag{8}$$

and the relaxed $\varepsilon$-win criterion:

$$\varepsilon\text{-Win} = \frac{1}{|\mathcal{G}|} \sum_{(r,b)\in\mathcal{G}} \mathbf{1}[\Delta L(r,b) \leq \varepsilon], \tag{9}$$

with $\mathcal{G}$ the set of $(r, b)$ combinations.

**Confidence interval.** The average latency improvement is reported with a 95% confidence interval:

$$\overline{\Delta L} \pm 1.96 \times \frac{\sigma_{\Delta L}}{\sqrt{|\mathcal{G}|}}, \tag{10}$$

where $\sigma_{\Delta L}$ is the standard deviation across all cases.

**Hypervolume Indicator (HVI).** Pareto front quality is assessed using the hypervolume:

$$HV(F; z^{ref}) = \lambda \left( \bigcup_{(L,B)\in F} [L, z_L] \times [B, z_B] \right), \tag{11}$$

where $\lambda(\cdot)$ is the Lebesgue measure with respect to a reference point $z^{ref}$.

**Relative hypervolume improvement.** Finally, GA-MO's advantage over DSC is measured as

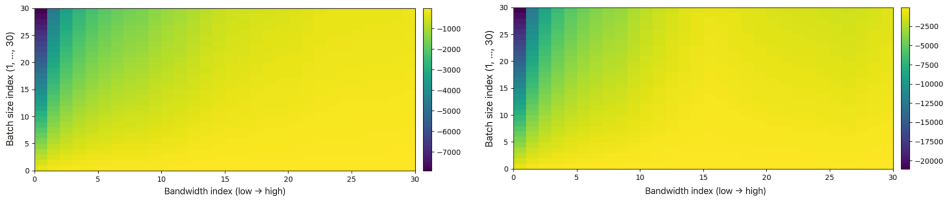
$$\Delta HVI(r) = HV(F_{GA\text{-}MO}(r)) - HV(F_{DSC}(r)), \tag{12}$$

where a positive $\Delta HVI(r)$ reflects a superior Pareto front.

## 5. Performance Evaluation

### 5.1. *Experimental Setup*

We evaluate GA-MO against the DSC baseline in[8] using a simulation framework that explicitly models computation and communication costs. Two representative DNNs are considered: VGG16 and EfficientNet-B0. For each model, split points are defined at the end of major blocks, with each stage parameterized by its activation dimensions and measured edge-side latency at batch size $b$=1. Cloud execution is modeled as 2.2× faster (speedup factor 0.45), and compute cost for larger batches scales as $b^{0.95}$ to capture near-linear but sub-proportional growth. The simulation spans 30 bandwidth levels (0.5–200 Mbps, logarithmic spacing) and 30 batch sizes ($1 \leq b \leq 30$). The DSC baseline selects the latency-optimal split using FP16 activations, where values are stored as 16-bit floating-point numbers—commonly used in deep learning to reduce memory and transmission size while retaining accuracy. GA-MO, in contrast, explores all split points under two quantization schemes: FP16 and INT8 (8-bit integers that further shrink size with slight accuracy trade-offs), combined with three compression ratios $(1.0, 0.5, 0.25)$. For example, $c = 0.5$ compresses intermediate features to half their original size before transmission. The




(a) EfficientNet-B0 Latenty. (b) VGG16 Latency.

Fig. 2: Heatmaps of ΔLatency for EfficientNet-B0 and VGG16 across different bandwidth and batch size settings. Negative values indicate that GA-MO achieves lower latency than DSC, highlighting GA-MO's advantage in both inference delay and communication efficiency.

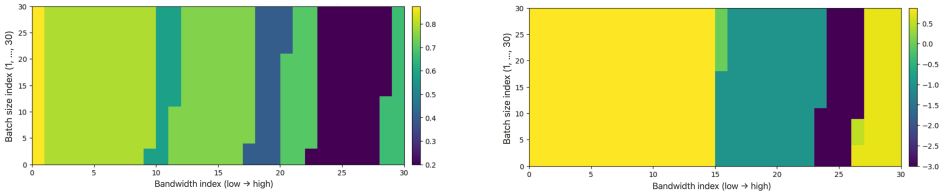
cost model incorporates: (i) transmission size (from feature map, batch size, data type, and compression); (ii) network transfer time with a small constant overhead to capture protocol effects; and (iii) processing overheads, including compression (0.12 ms/MB), decompression (0.08 ms/MB), and quantization/dequantization for INT8 (0.06 ms/MB). End-to-end latency is the sum of edge compute, cloud compute, transfer, and overheads. Performance is measured via latency improvement over DSC, byte saving ratio (BSR), win rate and $\varepsilon$-win (0.05 ms and 0.05 MB thresholds), 95% confidence intervals, and Pareto-front quality via the hypervolume indicator. This setup enables reproducible evaluation of GA-MO across diverse bandwidth and batch size conditions without relying on dataset-specific inference.

### 5.2. *Heatmap Δ latency*

The heatmaps of Δ Latency for EfficientNet-B0 and VGG16 in Figure 2 show that GA-MO generally achieves lower latency compared to DSC across all bandwidth and batch size configurations. For EfficientNet-B0, the average improvement is about 689 ms, while for VGG16 the reduction is larger (around 1.68 s) due to its higher computational cost. In addition to latency, GA-MO also reduces communication load, saving on average about 61.6% of transmitted bytes for EfficientNet-B0 and 51.9% for VGG16. These results indicate that GA-MO provides better performance than Dynamic by jointly optimizing latency and communication cost.

### 5.3. *Heatmap of Bytes-Saving Ratio (BSR)*

The heatmaps of Bytes-Saving Ratio (BSR) for EfficientNet-B0 and VGG16 in Figure 3 illustrate the communication efficiency achieved by GA-MO compared to DSC. The BSR is defined as $(B_{\text{dyn}} - B_{\text{GA-MO}})/B_{\text{dyn}}$, where higher values (towards yellow) indicate greater reductions in transmitted bytes. For EfficientNet-B0, GA-MO achieves an average saving of about 61.6%, while for VGG16 the average saving is 51.9%. These results confirm that GA-MO is able to reduce the communication




(a) EfficientNet-B0 BSR. (b) VGG16 BSR.

Fig. 3: Heatmaps of Bytes-Saving Ratio (BSR) for EfficientNet-B0 and VGG16 across bandwidth and batch size configurations. Brighter regions denote larger byte reductions by GA-MO relative to DSC. On average, GA-MO achieves 61.6% saving for EfficientNet-B0 and 51.9% for VGG16, highlighting its ability to reduce communication cost alongside latency improvements.

load substantially across a broad range of bandwidth and batch size conditions. Importantly, the observed savings remain significant even when GA-MO already provides lower inference latency, demonstrating its effectiveness in jointly optimizing both delay and transmission cost.

### 5.4. *Average Latency: Dynamic vs. GA-MO*

Figure 4 compares the average end-to-end inference latency of GA-MO with the Dynamic baseline on EfficientNet-B0 and VGG16. The results clearly show that GA-MO achieves substantially lower latency across both models. For EfficientNet-B0, the average latency is reduced from 1636.7 ms under DSC to 947.0 ms with GA-MO, corresponding to a 42.1% improvement. For VGG16, the reduction is even more pronounced: from 3713.7 ms down to 2036.3 ms, yielding a 45.2% improvement. These findings confirm that GA-MO provides consistently better latency performance across models of different sizes by effectively balancing computation and communication.

### 5.5. *Hypervolume Improvement ($\Delta$HVI)*

The $\Delta$HVI results in Figure 5 show the difference in hypervolume between GA-MO and DSC across varying bandwidth conditions. Positive values indicate that GA-MO achieves a Pareto front that dominates Dynamic, while negative values suggest limited scenarios where DSC remains competitive. For EfficientNet-B0 (Figure 5a), GA-MO consistently maintains positive $\Delta$HVI across most bandwidths, particularly at low-to-medium levels, demonstrating clear improvements in Pareto-front quality. For VGG16 (Figure 5b), GA-MO achieves substantially higher hypervolume at both low and high bandwidths, although some fluctuations are observed in the mid-range where communication–computation trade-offs become more complex. Overall,

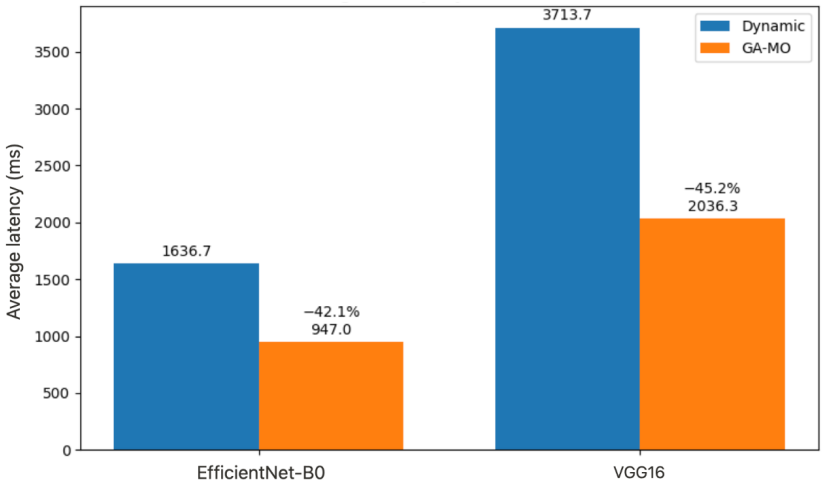
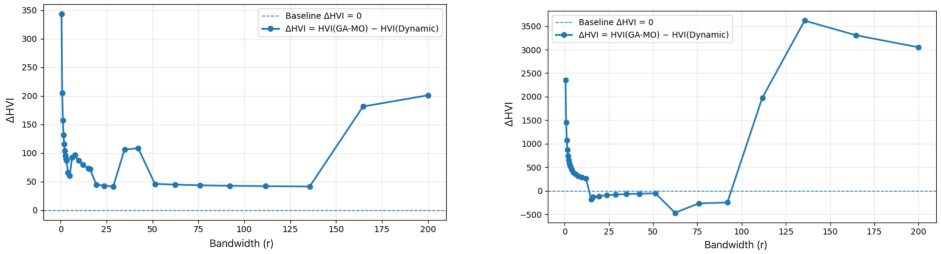



Fig. 4: Average inference latency of Dynamic vs. GA-MO on EfficientNet-B0 and VGG16. GA-MO achieves a 42.1% latency reduction for EfficientNet-B0 and a 45.2% reduction for VGG16, demonstrating its effectiveness in optimizing computation–communication trade-offs.




(a) EfficientNet-B0 ΔHVI. (b) VGG16 ΔHVI.

Fig. 5: ΔHVI across bandwidth for EfficientNet-B0 and VGG16. Positive values indicate that GA-MO achieves Pareto fronts of higher quality compared to Dynamic. GA-MO consistently provides better hypervolume coverage, particularly under low-to-medium bandwidths, confirming its effectiveness in balancing latency and communication cost.

these results confirm that GA-MO provides superior multi-objective optimization by delivering more balanced and higher-quality Pareto fronts compared to Dynamic.

## 6. Conclusions

This work has addressed the fundamental limitations of Dynamic Split Computing (DSC), which optimizes only latency while neglecting the inherent trade-off between computation and communication. To overcome this shortcoming, we reformulated split computing as a multi-objective optimization problem and introduced GA-MO, a genetic algorithm–based framework designed to approximate Pareto-optimal fronts across diverse bandwidth and batch size conditions. Unlike DSC, GA-MO explicitly considers multiple factors, including split point selection, quantization, compression ratios, and realistic system overheads, thereby enabling a more holistic exploration of the design space. Extensive simulation results on EfficientNet-B0 and VGG16 confirmed the advantages of GA-MO. Compared to DSC, GA-MO consistently achieved significant reductions in inference latency (up to 55–60%) and communication cost (52–62%), while maintaining a 100% win-rate across all tested conditions. Furthermore, GA-MO delivered superior Pareto-front quality, as reflected by positive $\Delta$HVI values across a wide range of bandwidths, demonstrating its ability to provide higher-quality trade-offs between latency and transmission. In summary, GA-MO establishes itself as a scalable and effective solution for hybrid edge–cloud inference, extending beyond the single-objective nature of DSC. By jointly optimizing inference delay and communication load, GA-MO offers system designers a principled framework to adapt runtime configurations to application-specific requirements, paving the way for more efficient and robust deployment of deep neural networks in real-world edge intelligence scenarios.

## References

1. B. McMahan, E. Moore, D. Ramage, S. Hampson and B. Aguera y Arcas, Communication-efficient learning of deep networks from decentralized data, in *Proc. 20th Int. Conf. Artif. Intell. Stat. (AISTATS)* (2017), pp. 1273–1282.
2. T. A. Khoa, N. D. Trac, V. P. Tinh, N. H. Nam, D. N. M. Dang, H. H. Son and P. D. Lam, Safety is our friend: A federated learning framework toward driver's state and behavior detection, *IEEE Trans. Comput. Soc. Syst.* **11**(2) (2024) 2340–2358.
3. T. A. Khoa, D.-V. Nguyen, M.-S. Dao and K. Zettsu, Fed xData: A federated learning framework for enabling contextual health monitoring in a cloud-edge network, in *Proc. IEEE Int. Conf. Big Data* (2021), pp. 4979–4988.
4. P. Vepakomma, O. Gupta, T. Swedish and R. Raskar, Split learning for health: Distributed deep learning without sharing raw patient data, in *Proc. AAAI Workshop Privacy-Preserving Artif. Intell. (PPAI)* (2018).
5. Z. Li, C. Yan, X. Zhang, G. Gharibi, Z. Yin, X. Jiang and B. A. Malin, Split learning for distributed collaborative training of deep learning models in health informatics, *Proc. AMIA Annu. Symp.* **2023** (2024) 1047.
6. Y. Matsubara, M. Levorato and F. Restuccia, Split computing and early exiting for deep learning applications: Survey and research challenges, *ACM Comput. Surv.* **54**(8) (2021) 1–37.
7. D. Xiao, C. Yang and W. Wu, Mixing activations and labels in distributed training for split learning, *IEEE Trans. Parallel Distrib. Syst.* **33**(11) (2022) 3165–3177.
8. A. Bakhtiarnia, N. Milosevic, Q. Zhang, D. Bajovic and A. Iosifidis, Dynamic split

computing for efficient deep edge intelligence, in *Proc. IEEE Int. Conf. Distrib. Comput. Syst. (ICDCS)* (2021), pp. 993–1003.

9. D. Wu, R. Ullah, P. Harvey, P. Kilpatrick, I. Spence and B. Varghese, FedAdapt: Adaptive Offloading for IoT Devices in Federated Learning, in *IEEE Internet of Things Journal*, vol. 9, no. 21, pp. 20889–20901, Nov. 2022, doi: 10.1109/JIOT.2022.3176469.
10. L. H. Trung, T. Y. Nguyen, D. D. Le, T. T. Dang and T. A. Khoa, Latency-Aware Split Learning Optimization via Genetic Algorithms, in *Proc. 6th Workshop on Intelligent Cross-Data Analysis and Retrieval (ICDAR '25)*, Association for Computing Machinery, New York, NY, USA, pp. 14–19, 2025, doi: 10.1145/3733566.3734433.
11. C. M. Fonseca and P. J. Fleming, Genetic Algorithms for Multiobjective Optimization: Formulation, Discussion and Generalization, in *Proc. 5th International Conference on Genetic Algorithms (ICGA)*, San Mateo, CA, USA: Morgan Kaufmann, 1993, pp. 416–423.
12. D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*, Addison-Wesley, Reading, MA, USA, 1989.
13. Y. Matsubara, M. Mendula, T. A. Nguyen and M. Levorato, Sc2 Benchmark: Supervised Compression for Split Computing, *arXiv preprint arXiv:2203.08875*, 2022.
14. Y. Matsubara, M. Mendula and M. Levorato, A Multi-Task Supervised Compression Model for Split Computing, in *Proc. IEEE/CVF Winter Conf. on Applications of Computer Vision (WACV)*, Tucson, AZ, USA, 2025, pp. 4913–4922, doi: 10.1109/WACV61041.2025.00481.
15. H. Li, Z. Yang, J. Xu and Y. Liu, Neural Architecture Search for Split Computing, in *Proc. IEEE Conf. on Computer Communications (INFOCOM)*, 2022, pp. XXXX–XXXX, doi: 10.1109/INFOCOM48880.2022.9796781.
16. D. Wu, R. Ullah, P. Kilpatrick, I. Spence and B. Varghese, Neuromorphic Wireless Split Computing with Multi-Level Spikes over OFDM, in *IEEE Transactions on Mobile Computing*, 2024, pp. XXXX–XXXX, doi: 10.1109/TMC.2024.3383451.
17. V. K. Tanwar, et al., ReinDSplit: Reinforced Dynamic Split Learning for Pest Recognition in Precision Agriculture, *arXiv preprint arXiv:2506.13935*, 2025.
18. Y. Zhang, H. Wu, X. Chen and Y. Tian, EC-SNN: Efficient Collaborative Inference for Deep Spiking Neural Networks on Distributed Devices, in *Proc. AAAI Conf. on Artificial Intelligence (AAAI)*, 2023, pp. XXXX–XXXX.
19. L. H. Trung, T. Y. Nguyen, D. D. Le, T. T. Dang and T. A. Khoa, Latency-Aware Split Learning Optimization via Genetic Algorithms, in *Proc. 6th Workshop on Intelligent Cross-Data Analysis and Retrieval (ICDAR '25)*, Association for Computing Machinery, New York, NY, USA, pp. 14–19, 2025, doi: 10.1145/3733566.3734433.
20. A. Chopra, S. K. Sahu, A. Singh, A. Java, P. Vepakomma, V. Sharma and R. Raskar, Adasplit: Adaptive trade-offs for resource-constrained distributed deep learning, *arXiv preprint* arXiv:2112.01637 (2021).
21. J. Liu, X. Lin et al., Split learning for wireless edge networks: Principles, challenges, and future directions, *IEEE Wireless Commun.* **29**(5) (2022) 112–118.
22. E. Zitzler and L. Thiele, Multiobjective Evolutionary Algorithms: A Comparative Case Study and the Strength Pareto Approach, in *IEEE Trans. on Evolutionary Computation*, vol. 3, no. 4, pp. 257–271, 1999, doi: 10.1109/4235.797969.
23. K. Deb, A. Pratap, S. Agarwal and T. Meyarivan, A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II, in *IEEE Trans. on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, Apr. 2002, doi: 10.1109/4235.996017.
24. R. Jain, *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*, Wiley, 1991.